

FORTRAN 90

Funciones y Subrutinas

Fortran 90 tiene dos tipos de subprogramas, las funciones y las subrutinas. Una función en Fortran 90 es igual a una función en cualquier otro lenguaje. Así una *función* regresa un solo valor calculado, vía el nombre de la función. Si necesitamos que un subprograma regrese más de un valor o inclusive que no regrese nada, usamos una *subrutina*.

Una función en Fortran o un subprograma tipo función, tiene la siguiente sintaxis:

```
type FUNCTION function-name (arg1, arg2, ..., argn)
IMPLICIT NONE
[specification part]
[execution part]
[subprogram part]
END FUNCTION function-name
```

Donde: **type** es un tipo en Fortran 90 (e.g., **INTEGER**, **REAL**, **LOGICAL**, etc) con o sin **KIND**.

function-name es un identificador en Fortran 90

arg1, ..., argn son los *argumentos* de la función.

Una función es una unidad o programa auto-contenido que recibe una entrada o “input” del programa principal vía sus *argumentos formales* de la función, este subprograma toma estas entradas y hace los cálculos, regresando su resultado en el nombre de la función. Es decir en alguna parte de la función se deben tener una o más instrucciones de asignación como esta:

function-name = *expression*

Donde el resultado de la *expression* es guardada en la variable con el nombre de la función. Se debe notar que la **function-name** no puede aparecer a la derecha en ninguna expresión.

Durante la especificación del tipo en la zona de declaraciones de los argumentos formales deberá haber un nuevo atributo el **INTENT (IN)**.

El significado de **INTENT (IN)** es tal que los valores que toma la función en sus argumentos formales no cambian su contenido.

Cualquier instrucción que es usada en **PROGRAM** o **programa principal**

puede se usada en una **FUNCTION**.

Es posible que las funciones no reciban ningún argumento formal. Pero aún así el **()** es requerido.

Cálculo del Factorial

```
INTEGER FUNCTION Factorial(n)
IMPLICIT NONE
INTEGER, INTENT(IN) :: n
INTEGER :: i, Ans
Ans = 1
DO i = 1, n
  Ans = Ans * i
END DO
Factorial = Ans
END FUNCTION Factorial
```

Read and return a positive real number

```
REAL FUNCTION GetNumber()
IMPLICIT NONE
REAL :: Input_Value
DO
  WRITE(*,*) 'A positive number: '
  READ(*,*) Input_Value
  IF (Input_Value > 0.0) EXIT
  WRITE(*,*) 'ERROR. try again.'
END DO
GetNumber = Input_Value
END FUNCTION GetNumber
```

Errores mas comunes al usar funciones:

- Olvidar el tipo de la función
- Olvidar **INTENT (IN)**
- Cambiar el argumento que tiene el **INTENT (IN)**

Ejemplo:

```
REAL FUNCTION DoSomething(a, b)
IMPLICIT NONE
INTEGER, INTENT(IN) :: a, b
IF (a > b) THEN
  a = a - b ; ←Error a no debería cambiar
ELSE
  a = a + b
END IF
DoSomething = SQRT(a*a+b*b)
END FUNCTION DoSomething
```

- Olvidar regresar un valor
- Incorrecto uso del nombre de la función

Ejemplo:

```
REAL FUNCTION DoSomething(a, b)
IMPLICIT NONE
```

```

INTEGER, INTENT(IN) :: a, b
DoSomething = a*a + b*b
DoSomething = SQRT(DoSomething)
END FUNCTION DoSomething

```

- Solo la última asignación es regresada

Ejemplo:

```

REAL FUNCTION DoSomething(a, b)
IMPLICIT NONE
INTEGER, INTENT(IN) :: a, b
DoSomething = a*a + b*b
DoSomething = SQRT(a*a - b*b)
END FUNCTION DoSomething

```

El uso de una función definida por el usuario es similar al uso de funciones intrínsecas de Fortran 90. Los siguientes usos de la función `Factorial(n)` para el cálculo del coeficiente combinatorio $C(m,n)$, donde m y n son los *argumentos* demuestra la utilidad de las funciones:

`Cmn = Factorial(m) / (Factorial(n) * Factorial(m-n))`

Nota: el coeficiente combinatorio es de la forma:

$$C(n, m) = \frac{m!}{n! \times (m - n)!}$$

Asociación de argumentos

La asociación de argumentos es el modo en que los argumentos que van a pasar de donde es invocada la función con sus “argumentos actuales” a la función misma con sus “argumentos formales”.

<pre> WRITE(*,*) Sum(a,b,c) . . . INTEGER FUNCTION Sum(x,y,z) IMPLICIT NONE INTEGER, INTENT(IN) :: x,y,z END FUNCTION Sum </pre>	a ↓ x	b ↓ y	c ↓ z
---	---	---	---

Si un argumento actual es una *expression*, está primero es evaluada *guardada en una variable temporal* de la cual el valor será pasado al correspondiente argumento formal, ejemplo:

```

WRITE(*,*) Sum(a+b,b+c,c)
.
.
.

INTEGER FUNCTION Sum(x,y,z)
IMPLICIT NONE
INTEGER, INTENT(IN) :: x,y,z
.....
END FUNCTION Sum

```

En caso de tener constantes como argumentos actuales, ejemplo:

```

WRITE(*,*) Sum(1,2,3)
.
.
.

INTEGER FUNCTION Sum(x,y,z)
IMPLICIT NONE
INTEGER, INTENT(IN) :: x,y,z
.....
END FUNCTION Sum

```

En Fortran 90 las funciones pueden ser internas o externas. Las funciones *Internas* están adentro del **PROGRAM**, así tenemos una nueva estructura de un programa el *main program* se ve así:

```

PROGRAM program-name
IMPLICIT NONE
[specification part]
[execution part]
CONTAINS
[functions]
END PROGRAM program-name

```

Aunque una function puede contener a otras funciones una función interna **no puede** contener otra función interna. A continuación mostramos un ejemplo con dos funciones internas **ArithMean()** y **GeoMean()**. Estas dos funciones toman dos argumentos reales y regresan un valor de tipo real.

```

PROGRAM TwoFunctions
  IMPLICIT NONE
  REAL :: a, b, A_Mean, G_Mean
  READ(*,*) a, b
  A_Mean = ArithMean(a, b)
  G_Mean = GeoMean(a,b)
  WRITE(*,*) a, b, A_Mean, G_Mean
CONTAINS

  REAL FUNCTION ArithMean(a, b)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b
    ArithMean = (a+b)/2.0
  END FUNCTION ArithMean

  REAL FUNCTION GeoMean(a, b)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b
    GeoMean = SQRT(a*b)
  END FUNCTION GeoMean

END PROGRAM TwoFunctions

```

Variables Globales y locales

Las “*Scope rules*” nos dicen si una entidad (*i.e.*, variable, parámetro o función) es *visible* o *accesible* en ciertos lugares. El lugar donde una entidad puede ser accesible o visible es referida como el alcance o “*scope*” de una entidad.

Regla de alcance o Scope: El alcance o scope de una entidad es el programa o función en el cual está declarada.

```

PROGRAM Scope_1
  IMPLICIT NONE
  REAL, PARAMETER :: PI = 3.1415926
  INTEGER :: m, n
  .....
CONTAINS

  INTEGER FUNCTION Funct1(k)
    IMPLICIT NONE
    INTEGER, INTENT(IN) :: k
    REAL :: f, g
    .....
  END FUNCTION Funct1

  REAL FUNCTION Funct2(u, v)
    IMPLICIT NONE
    REAL, INTENT(IN) :: u, v
    .....
  END FUNCTION Funct2

END PROGRAM Scope_1

```

“Scope” de
Pi, m y n

“Scope” de
k, f y g
“locales” a
la función
Funct1()

“Scope” de
u y v
“locales” a
Funct2()

Segunda regla de scope o alcance: Una variable o entidad **global** es **visible** a todas las funciones dentro de un contains.

```
PROGRAM Scope_2
IMPLICIT NONE
INTEGER :: a = 1, b = 2, c = 3
WRITE(*,*) Add(a)
c = 4
WRITE(*,*) Add(a)
WRITE(*,*) Mul(b,c)
CONTAINS
```

```
INTEGER FUNCTION Add(q)
IMPLICIT NONE
INTEGER, INTENT(IN) :: q
Add = q + c
END FUNCTION Add
```

```
INTEGER FUNCTION Mul(x, y)
IMPLICIT NONE
INTEGER, INTENT(IN) :: x, y
Mul = x * y
END FUNCTION Mul
```

```
END PROGRAM Scope_2
```

- **a, b y c** son globales
- El primer **Add(a)** **regresa 4**
- El **segundo Add(a)** **regresa 5**
- **Mul(b, c)** **regresa 8**

Así, los dos **Add(a)**'s producen diferentes resultados, ¡incluso si los argumentos formales son los mismos!

Otro ejemplo de la regla 2: Una variable **global** es **visible** a todas las funciones en un "contains".

```
PROGRAM Global
IMPLICIT NONE
INTEGER :: a=10, b=20
WRITE(*,*) Add(a,b)
WRITE(*,*) b
WRITE(*,*) Add(a,b)
CONTAINS
```

```
INTEGER FUNCTION Add(x,y)
IMPLICIT NONE
INTEGER, INTENT(IN) :: x, y
b = x+y
Add = b
END FUNCTION Add
```

```
END PROGRAM Global
```

- El primer **Add(a,b)** **regresa 30**
- Esto también **cambia a b a 30**
- El Segundo **WRITE(* *)** **muestra 30**
- El segundo **Add(a,b)** **regresa 40**

Regla de alcance o scope 3: Una entidad declarada en el “scope” de otra entidad es siempre diferente incluso si sus nombres son iguales.

```
PROGRAM Scope_3
IMPLICIT NONE
INTEGER :: i, Max = 5
DO i = 1, Max
Write(*,*) Sum(i)
END DO
CONTAINS
```

```
INTEGER FUNCTION Sum(n)
IMPLICIT NONE
INTEGER, INTENT(IN) :: n
INTEGER :: i, s
s = 0
..... other computation .....
Sum = s
END FUNCTION Sum
```

```
END PROGRAM Scope_3
```

Aunque **PROGRAM** y **FUNCTION Sum()** tienen ambos la variable **i** de tipo **INTEGER**, Estas serán dos entidades *are diferentes*.

Entonces, cualquier cambio de **i** en **Sum()** no afectará al **i** del **PROGRAM**.

Ejemplo:

Si un triángulo tiene lados de longitud los lados **a**, **b** y **c**, la fórmula de Heron calcula el área del triángulo como sigue, donde $s = (a+b+c)/2$:

$$Area = \sqrt{s \times (s - a) \times (s - b) \times (s - c)}$$

Para formar un triángulo **a**, **b** y **c** deberán cumplir las siguientes dos condiciones:

- $a > 0$, $b > 0$ y $c > 0$
- $a+b > c$, $a+c > b$ y $b+c > a$

La función **LOGICAL TriangleTest()** asegura que todos los lados sean positivos, y que la suma de cualesquiera dos lados sean mas grandes que el tercero.

```
LOGICAL FUNCTION TriangleTest(a, b, c)
IMPLICIT NONE
REAL, INTENT(IN) :: a, b, c
LOGICAL :: test1, test2
test1 = (a > 0.0) .AND. (b > 0.0) .AND. (c > 0.0)
test2 = (a + b > c) .AND. (a + c > b) .AND. (b + c > a)
TriangleTest = test1 .AND. test2 ! ambas condiciones
                                ! deberan ser verdaderas
END FUNCTION TriangleTest
```

Esta función a continuación implementa la fórmula de Heron. Hay que notar **a**, **b** y **c** deberá formar un triángulo.

```
REAL FUNCTION Area(a, b, c)
IMPLICIT NONE
REAL, INTENT(IN) :: a, b, c
REAL :: s
```

```

s = (a + b + c) / 2.0
Area = SQRT(s*(s-a)*(s-b)*(s-c))
END FUNCTION Area

```

El programa principal se vería:

```

PROGRAM HeronFormula
IMPLICIT NONE
REAL :: a, b, c, TriangleArea
DO
  WRITE(*,*) 'Three sides of a triangle please --> '
  READ(*,*) a, b, c
  WRITE(*,*) 'Input sides are ', a, b, c
  IF (TriangleTest(a, b, c)) EXIT ! exit if they form a triangle
  WRITE(*,*) 'Your input CANNOT form a triangle. Try again'
END DO
TriangleArea = Area(a, b, c)
WRITE(*,*) 'Triangle area is ', TriangleArea
CONTAINS
  LOGICAL FUNCTION TriangleTest(a, b, c)
    .....
  END FUNCTION TriangleTest
  REAL FUNCTION Area(a, b, c)
    .....
  END FUNCTION Area
END PROGRAM HeronFormula

```

Subrutinas

Una función en Fortran 90 toma valores de sus argumentos formales, y regresa un *solo valor* usando el nombre de la función.

En una subrutina en Fortran 90 toma valores de sus argumentos formales y *regresa algunos valores calculado también en sus argumentos formales*. Una subrutina en Fortran 90 no devuelve ningún valor con su nombre.

La sintaxis de una subrutina en Fortran 90 tiene la siguiente forma:

```

SUBROUTINE subroutine-name(arg1,arg2,...,argn)
IMPLICIT NONE
[specification part]
[execution part]
[subprogram part]
END SUBROUTINE subroutine-name

```

Si una función no requiere ningún argumento formal “arg1, arg2, . . . , argn” solo se colocan los (). Entonces las subrutinas usan argumentos formales para recibir y pasar resultados de regreso, entonces junto con **INTENT (IN)**, habrá también un **INTENT (OUT)** y un **INTENT (INOUT)**. **INTENT (OUT)** significa un argumento formal que no recibe un valor pero esté regresará un valor a su correspondiente argumento actual. **INTENT (INOUT)** significa que el argumento formal recibe un valor y regresa un valor a su correspondiente argumento actual.

Dos ejemplos simples:

```
SUBROUTINE Means(a, b, c, Am, Gm, Hm)
  IMPLICIT NONE
  REAL, INTENT(IN) :: a, b, c
  REAL, INTENT(OUT) :: Am, Gm, Hm
  Am = (a+b+c)/3.0
  Gm = (a*b*c)**(1.0/3.0)
  Hm = 3.0/(1.0/a + 1.0/b + 1.0/c)
END SUBROUTINE Means
```

**Am, Gm y Hm son
usados para
regresar los
resultados**

**Los valores de
a y b son
intercambiados**

```
SUBROUTINE Swap(a, b)
  IMPLICIT NONE
  INTEGER, INTENT(INOUT) :: a, b
  INTEGER :: c
  c = a
  a = b
  b = c
END SUBROUTINE Swap
```

Para usar una subrutina en Fortran 90 se utiliza la instrucción **CALL** es necesaria. La instrucción **CALL** puede tener alguna de estas formas:

- **CALL sub-name(arg1,arg2,...,argn)**
- **CALL sub-name()**
- **CALL sub-name**

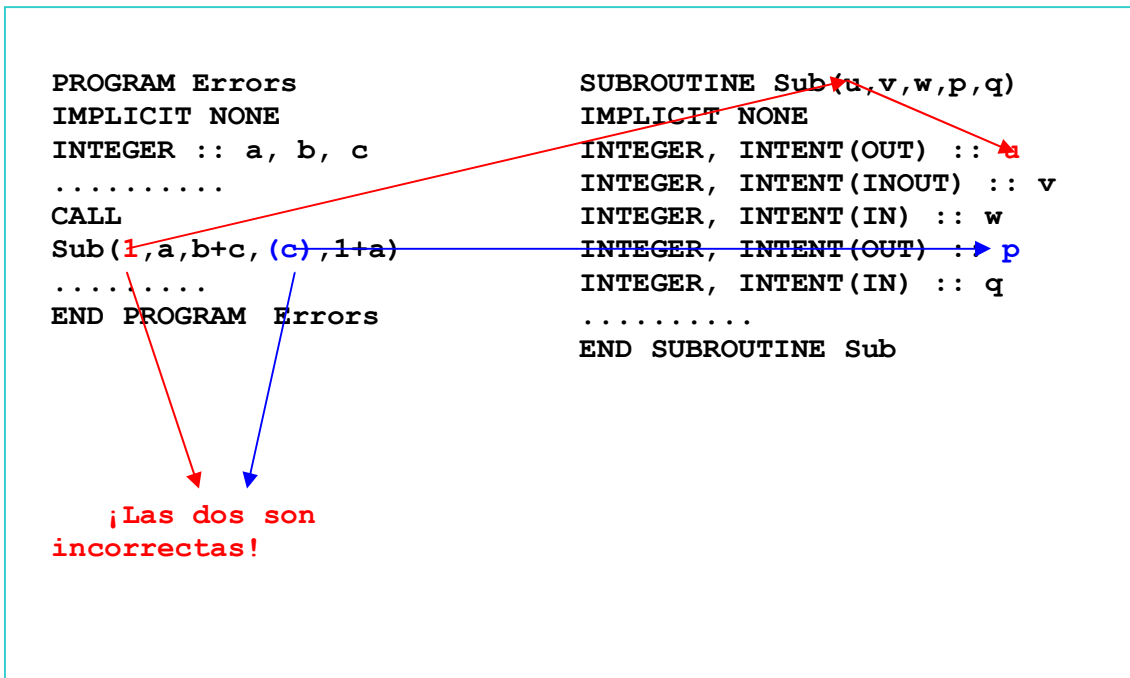
Las últimas dos formas son equivalentes y son para llamar a una subrutina sin argumentos formales.

```
PROGRAM Test
  IMPLICIT NONE
  REAL :: a, b
  READ(*,*) a, b
  CALL Swap(a,b)
  WRITE(*,*) a, b
  CONTAINS
  SUBROUTINE Swap(x,y)
    IMPLICIT NONE
    REAL, INTENT(INOUT) :: x,y
    REAL :: z
    z = x
    x = y
    y = z
  END SUBROUTINE Swap
END PROGRAM Test
```

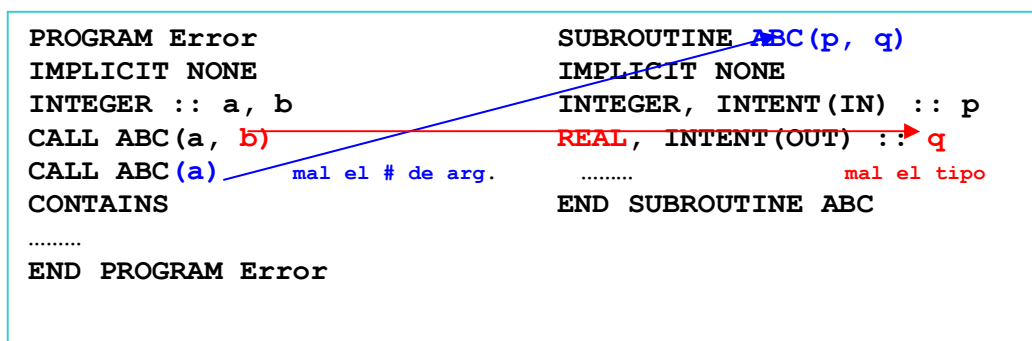
```
PROGRAM SecondDegree
  IMPLICIT NONE
  REAL :: a, b, c, r1, r2
  LOGICAL :: OK
  READ(*,*) a, b, c
  CALL Solver(a,b,c,r1,r2,OK)
  IF (.NOT. OK) THEN
    WRITE(*,*) "No root"
  ELSE
    WRITE(*,*) a, b, c, r1, r2
  END IF
  CONTAINS
  SUBROUTINE Solver(a,b,c,x,y,L)
    IMPLICIT NONE
    REAL INTENT(IN) :: a,b,c
    REAL, INTENT(OUT) :: x, y
    LOGICAL, INTENT(OUT) :: L
    .....
  END SUBROUTINE Solver
END PROGRAM SecondDegree
```

Más acerca de la asociación de argumentos

Entonces, si un argumento formal con el atributo `INTENT(OUT)` o `INTENT(INOUT)` puede pasar valores de regreso al correspondiente argumento, el *argumento actual deberá ser una variable*. Veamos estos errores



El número de argumentos y el tipo de estos deberán coincidir en el call y en la definición de la subrutina. Lamentablemente no hay una conversión de tipos entre argumentos veamos el siguiente error:



Módulos enFortran 90

Es posible en F90 poner todas las funciones y subrutinas relevantes juntas en algo que se llama un modulo. Un modulo, en un lenguaje OO es posible verlo cercano a una clase estática con información pública y privada y sus métodos.

Así en algún sentido un módulo en Fortran 90 module provee algo de conceptos de programación orientada a objetos.

Un modulo en Fortran 90 tiene la siguiente sintaxis:

```
MODULE module-name
IMPLICIT NONE
[specification part]
CONTAINS
[internal functions/subroutines]
END MODULE module-name
```

En la parte de “specification part” e “internal functions and Subroutines” son opcionales. Un modulo puede verse como un **PROGRAM**, excepto que no tiene una parte ejecutable. Así, un programa principal deberá estar presente para hacer uso de módulos.

Ejemplo:

El módulo `SomeConstants` no tiene la parte de subprogramas o sea no hay ni funciones ni subrutinas.

```
MODULE SomeConstants
  IMPLICIT NONE
  REAL, PARAMETER :: PI=3.1415926
  REAL, PARAMETER :: g = 980
  INTEGER :: Counter
END MODULE SomeConstants
```

El modulo `SumAverage` no tiene la perte de especificación.

```
MODULE SumAverage
CONTAINS
  REAL FUNCTION Sum(a, b, c)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b, c
    Sum = a + b + c
  END FUNCTION Sum
  REAL FUNCTION Average(a, b, c)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b, c
    Average = Sum(a,b,c)/2.0
  END FUNCTION Average
END MODULE SumAverage
```

-Un módulo generalmente tiene ambas partes la de **specification** y la parte de **internal Functions**.

-Normalmente, este es el caso.

```
MODULE DegreeRadianConversion
IMPLICIT NONE
REAL, PARAMETER :: PI = 3.1415926
REAL, PARAMETER :: Degree180 = 180.0
CONTAINS
```

```
REAL FUNCTION
DegreeToRadian(Degree)
IMPLICIT NONE
REAL, INTENT(IN) :: Degree
DegreeToRadian =
Degree*PI/Degree180
END FUNCTION DegreeToRadian
```

```
REAL FUNCTION RadianToDegree(radian)
IMPLICIT NONE
REAL, INTENT(IN) :: Radian
RadianToDegree= Radian*Degree180/PI
END FUNCTION RadianToDegree
```

```
END MODULE DegreeRadianConversion
```

Compilación de un Programa distribuido entre archivos

Supongamos que un programa consiste de un programa principal a **main.f90** y 2 módulos **Test.f90** y **Compute.f90**. En general, estos se pueden compilar de la siguiente manera:

```
f90 main.f90 Test.f90 Compute.f90 -o main
```

Sin embargo, en algunos compiladores puede resultar más restrictivo. *Se listan esos módulos que no usan otros módulos primero, seguido por esos módulos que solamente usan los módulos listados seguidos por el programa principal.*

Supongamos que tenemos los módulos **A, B, C, D** y **E**, y **C** usa a **A**, **D** usa **B**, y **E** usa **A, C** y **D**, entonces la forma más segura de compilar el programa completo es siguiendo el siguiente comando:

```
f90 A.f90 B.f90 C.f90 D.f90 E.f90 main.f90 -o main
```

Supongamos que tenemos los módulos **A, B, C, D** y **E**, y **C** usa **A, D** usa **B**, y **E** usa **A, C** y **D**. Pensando que los módulos son desarrollados separadamente con alguna funcionalidad específica en mente es posible compilar cada módulo a un código objeto como sigue:

```
f90 -c A.f90
f90 -c B.f90
f90 -c C.f90
f90 -c D.f90
f90 -c E.f90
```

Hay que notar que el orden todavía es importante. Lo de arriba genera los siguientes archivos **A.o, B.o, C.o, D.o** y **E.o**

Si un programa principal se encuentra en el archivo `prog2.f90` y usa los módulos en `A.f90` y `B.f90`, es posible compilar y generar un código ejecutable `prog2` como sigue:
`f90 A.o B.o prog2.f90 -o prog2`

Ejemplos:

El coeficiente combinatorial de m y n ($m \geq n$) es $C_{m,n} = m!/(n! \cdot (m-n)!)$.

```
MODULE FactorialModule
  IMPLICIT NONE
  CONTAINS
    INTEGER FUNCTION Factorial(n)
      IMPLICIT NONE
      INTEGER, INTENT(IN) :: n
      ... other statements ...
    END FUNCTION Factorial

    INTEGER FUNCTION Combinatorial(n, r)
      IMPLICIT NONE
      INTEGER, INTENT(IN) :: n, r
      ... other statements ...
    END FUNCTION Combinatorial
  END MODULE FactorialModule
```

```
PROGRAM ComputeFactorial
  USE FactorialModule
  IMPLICIT NONE
  INTEGER :: N R
  READ(*,*) N, R
  WRITE(*,*) Factorial(N)
  WRITE(*,*) Combinatorial(N,R)
END PROGRAM ComputeFactorial
```

`Combinatorial(n,r)`
 usa `Factorial(n)`

Otro ejemplo:

Funciones trigonométricas usando grados.

```
MODULE TrigonometricFunctions
  IMPLICIT NONE
  REAL, PARAMETER :: PI = 3.1415926
  REAL, PARAMETER :: Degree180 = 180.0
  REAL, PARAMETER :: R_to_D = Degree180/PI
  REAL, PARAMETER :: D_to_R = PI/Degree180
  CONTAINS
```

```
  REAL FUNCTION DegreeToRadian(Degree)
    IMPLICIT NONE
    REAL, INTENT(IN) :: Degree
    DegreeToRadian = Degree * D_to_R
  END FUNCTION DegreeToRadian
```

```
  REAL FUNCTION MySIN(x)
    IMPLICIT NONE
    REAL, INTENT(IN) :: x
    MySIN = SIN(DegreeToRadian(x))
  END FUNCTION MySIN
```

```
  ... other functions ...
END MODULE TrigonometricFunctions
```

```
PROGRAM TrigonFuncTest
  USE TrigonometricFunctions
  IMPLICIT NONE
  REAL :: Begin = -180.0
  REAL :: Final = 180.0
  REAL :: Step = 10.0
  REAL :: x
  x = Begin
  DO
    IF x > Final) EXIT
    WRITE(*,*) MySIN(x)
    x = x + Step
  END DO
END PROGRAM TrigonFuncTest
```